

1st Assignment COMPX523-26A

Nick Lim, Albert Bifet

March 2026

1 Overall Description

This assignment aims to implement a modified version of the k Nearest Neighbours (kNN) classifier. This assignment has a value of 20% of the final score and has two parts, each of them graded as follows:

1. Coding and report (10% of the final score);
2. Debugging and analysis (10% of the final score).

Important information:

- **Deadline: See Moodle**
- Code the assignment preferably using either MOA or CapyMOA.
- There is no need to implement the algorithm from scratch; both MOA and CapyMOA already have implementations that can be extended ¹.
- **Depending on the chosen language, your submission will contain 2 or 4 files:**
 - **Python submissions will contain two (2) files:**
 1. One file containing the code (include both classes in the same .py);
 2. One .pdf file containing a write-up for the Section 5, and chat logs from LLMs if used.
 3. One .csv file with the required results (described in section 6.1);
 - **Java submissions will contain four (4) files:**
 1. Two .java files, one for each class;
 2. One .pdf file containing a write-up for the Section 5, and chat logs from LLMs if used.
 3. One .csv file with the required results (described in section 6.1);

Quick notes:

- Send only the necessary files for running your class (do not send the whole MOA folder, or a whole CapyMOA installation).
- Submissions without the code will score 0 on the **Coding** grade.
- Submissions with csv in the wrong format will also score 0 in the **Coding** grade.
- The use of LLMs are allowed, however chat logs are needed to be included in the submission.

¹For example, you can copy the kNN class in MOA and adapt it. For CapyMOA, a sample kNN class along with helper classes encapsulating the nitty-gritty details will be provided.

1.1 Evaluation procedure

Coding and report grade will be scored "automatically". First, the code will go through a similarity evaluation to detect codes that are too similar. Second, we will use a unit test to test for implementation issues, (i/e memory leaks, incorrect implementation of windowing, overflows, inability to handle multiple classes). To ensure that the unit test runs smoothly, use the following variable names as the class objects

- **window, class.window**: the KNN-windows for KNN.asw and KNN.cw
- **k**: number of neighbours
- **w**: window size

Third, the codes will be checked for reproducibility, set the random seed to your student ID. The results obtained in rerunning the experiments with the students' provided code should match the results provided in the csv file. Fourth, the write up will be marked against a rubric. Finally, the csv file will be checked against the "correct" results, indicating how correct the implementation is.

2 Coding

You can create a new classifier by implementing a sliding window kNN from scratch, or you can **create a copy**² and adapt one of the existing kNN implementations. However, we **strongly recommend** that you use the existing implementations. Observe that your kNN adaptation must be implemented in a way that it can be executed as a stand-alone classifier.

Standard kNN. A standard implementation of kNN for data streams maintains a single sliding window W with the latest N instances seen. The prediction $\hat{y}$ for an unlabelled instance x is the most common label among the k closest instances (nearest neighbors) to x in W , i.e. $neighbours(x, k, W)$. This process is shown in Figure 1, where all instances (red and blue circles) belong to W .

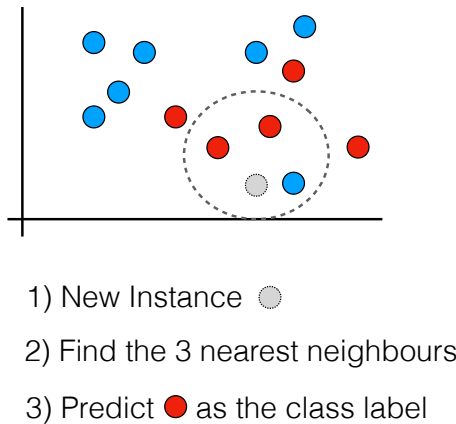


Figure 1: Majority vote using $k = 3$ for a binary classification 2 dimensional problem.

2.1 kNN with per class windowing (kNN-cw)

Training. In contrast to the standard kNN, you are asked to maintain c windows, where c corresponds to the number of class labels (Figure 2). Each window will contain a maximum of w instances³. After a window reaches w instances, it will start forgetting the oldest member of that window. Note that c may not be known ahead of time, and would need to increase as a new class is introduced. Notice that this

²You will need the original kNN implementation for the experiments, so do not override it

³This parameter is the same as the *limit* in MOA

is very similar to the standard kNN algorithm, except that the instance is stored in a specific window corresponding to its class label during training.

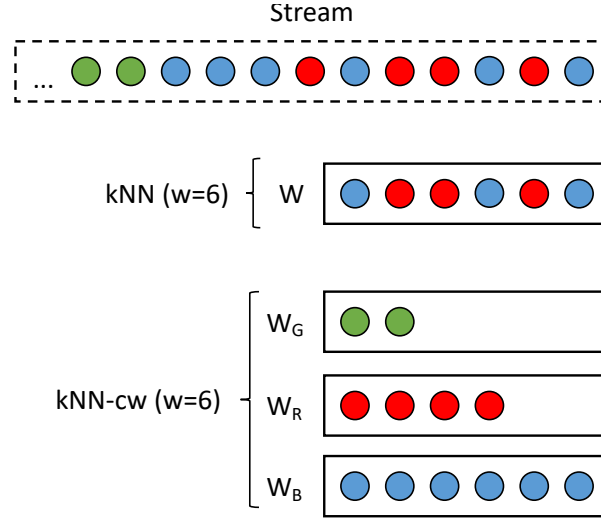


Figure 2: Comparison between standard sliding window kNN x sliding window kNN-cw. The parameter w determines the window size. The k parameter is not shown in this figure.

Predictions. A prediction will use all the windows, i.e. $\{W_0, W_1, \dots, W_c\}$. The distance from x will be calculated in each of the windows. After obtaining the k closest neighbours in each window, you will have a set of $class_number * k$ nearest neighbours. You should perform another search on this result to find the **overall nearest neighbours**. The final prediction is given by the majority vote⁴ of the **overall nearest neighbours**. The prediction is very similar to when we had only one sliding window to store all the instances. The main difference is that now the minority classes will have better representation.

2.2 kNN with age-weighted sampling window (kNN-asw)

Training. In this implementation of kNN, we return to a single-window implementation, however we keep track of the age of the sample α , where the age of all items in the window which increases by 1 every time a new instance arrives. We add the new instance to the window. If adding the instance causes the number of instances in the window to exceed w , one (and only one) member of the window is ejected with probability proportional to 1.2^α . Notice that this makes it such that the oldest instance is not always the instance that is ejected, and it is possible that the latest incoming instance is immediately ejected⁵⁶. Figure 3 illustrates this implementation visually.

Predictions. A prediction will use the standard KNN method. The distance from x will be calculated for each instance in the window. After obtaining the k closest neighbours in the window. The final prediction is given by the majority vote of the k **nearest neighbours**.

⁴majority = half of the votes cast + 1

⁵Note that you need to be careful with the implementation as blindly implementing will cause crashes and/or incorrect results.

⁶Hint: the largest number we can represent with 64 bits is $\approx 10^{308}$. Consider how else can you obtain a mathematically equivalent result.

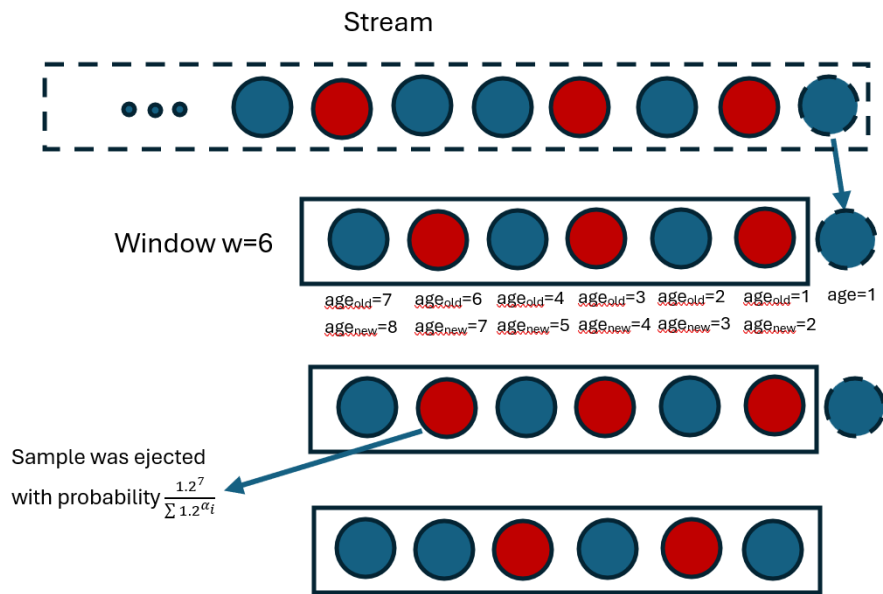


Figure 3: Age Weighted Sampling: observe that the ages of the instances are not in sequential order, and the oldest instance is not always the sample ejected from the window

3 Evaluation and Analysis

Evaluation framework. For all experiments, use a prequential evaluation with cumulative results⁷. That is, the result obtained after testing using all instances.

Do not present intermediate results, only the final cumulative result⁸.

3.1 Datasets

You are going to use the full electricity dataset and the first 50k instances from the RBFm_100k dataset. To use only the first 50k instances from this dataset, use CapyMOA's *max_instances* parameter and MOA's *instanceLimit* parameter when you are setting up the experiment.

- CapyMOA: Simple import and use standard framework datasets:

```
from capymoa.datasets import Electricity, RBFm_100k
```

- MOA: ARFF files are available in Moodle.

4 Experiments

Next are the instructions to generate the *csv* result file.

You have **three** algorithms: KNNBase (provided), KNN-cw, KNN-asw.

You have **two** datasets: Electricity and RBFm_10050k.

You will need to perform **four** experiments for **each algorithm-dataset combination**. For instance, you will run four different parameter configurations for KNN-cw on electricity dataset. The parameter configurations are the same for each algorithm-dataset pair:

1. $k=3$, window = 500
2. $k=21$, window = 500
3. $k=3$, window = 100
4. $k=21$, window = 100

In total you will have 3 algorithms, 2 dataset, 4 parameter configurations, which equals to **24 results**.

4.1 Guiding questions

The answers for such questions are not supposed to be submitted. They are questions that will help you reason about the experiments and understand implications of the parameters and design choices of the algorithms.

You are welcome to seek advice through email in case you want more details. If a large portion of students require discussion on the same question, that could be a topic for class discussion at any point.

1. What is the best model in terms of accuracy (i.e., highest accuracy)?
2. In general, has the performance improved with the addition of cw and asw when compared to base knn? Why do you think this happened?
3. Is there a predictive performance vs runtime performance trade-off when changing the window size? If yes, is it the same for both datasets? Are all algorithms equally affected?
4. Comparing the cw, asw. Are there trade-offs for using two techniques? How can we quickly adapt asw to maintain an approximately class-balanced window within a single window. How would changing the exponent affect the balance of the classes.

⁷**Watch out** for MOA, as the prequential evaluation task uses the windowed results by default

⁸You can achieve this by tuning the parameter in MOA's evaluation task or specifically accessing for cumulative results in CapyMOA

5 Questions and write up

1. Algorithm 1 is an LLM's implementation of a per class windowed KNN classifier⁹. Identify the trade-offs in this implementation. What are the advantages and drawbacks between your implementation KNN_cw implementation (from section 2.1) compared to this implementation?
2. Identify the changes we can introduce to your implementation of KNN_asw (from section 2.2) maintain an **approximately class-balanced** window with just a single window¹⁰.

6 Preparing your submission

6.1 CSV result file formatting

First line of the *csv* should contain the following column names (case sensitive):

- student_id (string, your student ID)
- dataset (string, rbf or electricity)
- knn_idenfifier (string, knn, knn-cw, knn-asw)
- window_size (integer, 100 or 500)
- k (integer, 3 or 21)
- accuracy (float, up to 3rd decimal)
- recall (float, up to 3rd decimal)
- precision (float, up to 3rd decimal)
- wallclock (float, seconds, up to 3rd decimal)

An empty csv file containing the correct header will be provided. This way you can focus on generating results and appending to the csv.

6.2 Code instructions

This is the expected naming stucture that is valid for both languages:

- KNN_cw_studentID
- KNN_asw_studentID

If you submit a Java code, your two files must adhere to the naming convention (consequently, the two classes as well).

For Python code, you should rename the provided `knnAssignemnt.py` into `assignment1.studentID.py` (if you have already named your file as `studentID.assignment1.py` that is fine). This file will contain the helper classes, the KNNBase class and the **two** new classes you implemented. For the **two** new knn versions, their class names and the corresponding `__str__` method should follow the convention.

6.3 General tips

One suggestion is that you learn how the standard kNN works before making any changes. Important things to look at are how the nearest neighbours happen, how the instances are stored in the window, how the window is initialized.

⁹A python implementation of this algorithm is available at the end of this document.

¹⁰Hint: the solution only requires 1 or 2 changes (if you want the class balance to be better preserved) from the current implementation.

Algorithm 1: Class-based Windowed K-Nearest Neighbors

```
1 Class LLMClassWindowKNN
2   Method INIT(k, w)
3     self.k  $\leftarrow k$  // number of neighbors
4     self.w  $\leftarrow w$  // window size per class
5     self.X  $\leftarrow$  empty list // feature vectors
6     self.y  $\leftarrow$  empty list // labels
7     self.class_window  $\leftarrow$  empty map // label  $\rightarrow$  list of indices

8   Method train(instance)
9     feat  $\leftarrow$  instance.feat
10    label  $\leftarrow$  instance.label
11    idx  $\leftarrow$  length(self.X)
12    append feat to self.X
13    append label to self.y
14    if label  $\notin$  self.class_window then
15      self.class_window[label]  $\leftarrow$  empty list
16    append idx to self.class_window[label]
17    // enforce per-class window size
18    if length(self.class_window[label])  $\geq$  self.w then
19      remove oldest index from self.class_window[label]

19  Method predict(instance)
20    topDistance  $\leftarrow$  empty list // to store (dist, label) tuples
21    // Step 1: Find k-nearest per class window
22    foreach label, indices  $\in$  self.class_window do
23      class_distances  $\leftarrow$  empty list
24      foreach idx  $\in$  indices do
25        d  $\leftarrow$  distance(self.X[idx], instance.feat)
26        append (d, label) to class_distances
27        // Store k smallest distances from this class
28        k_class_neighbors  $\leftarrow$  k smallest in class_distances by distance
29        topDistance.extend(k_class_neighbors)

30    // Step 2: Find global k smallest within topDistance
31    final_neighbors  $\leftarrow$  k smallest in topDistance by distance

32    // Step 3: Majority vote
33    votes  $\leftarrow$  empty map
34    foreach (dist, label)  $\in$  final_neighbors do
35      votes[label]  $\leftarrow$  votes[label] + 1
36    return label with maximum vote in votes
```

6.4 CapyMOA tips

A notebook that imports things given the correct folder structure is also being provided. All the methods that you need to modify are shown. `predict_proba` can be ignored, focus on **train** and **predict**.

6.5 MOA tips

In MOA, look for the kNN implementation available at **kNN.java**. For this assignment, you do not need to worry about the Regression part of the code (it is shown with comments).

In general, you will need to modify the following methods and attributes. If you decide to create more methods or modify another method, that is alright as well.

```
protected Instances window;  
public void setModelContext(InstancesHeader context)  
public void trainOnInstanceImpl(Instance inst)  
public double[] getVotesForInstance(Instance inst)
```

You can simply create an array of windows based on the amount of class labels. The method `setContext` is invoked prior to starting the learning process, so you can initialize the windows in it. Notice that the context object has a method that tells you the number of class labels, as shown below.

```
public void setModelContext(InstancesHeader context) {  
    int numberOfClasses = context.numClasses();  
}
```

To get started extending MOA code you can follow this tutorial [Building MOA from the source](#).

6.5.1 Evaluation

Make sure you have the Evaluator set as: **"BasicClassificationPerformanceEvaluator -o -r"**, otherwise you won't obtain the recall for each class on your output. A sample command that you can copy and paste in MOA GUI can be found below.

```
EvaluatePrequential -l trees.HoeffdingTree  
-s (ArffFileStream -f GMSC_nm.arff)  
-e (BasicClassificationPerformanceEvaluator -o -r)  
-i 150000 -f 100
```

Notice that **"-f 100"** does not influence the end result, it is just how often the intermediary results are printed. You can also set **"-f 50000"** so that you only get one row of results (the last one). If you are running it on the command line, you might use something like this:

```
java -Xmx4g -Xss10M -cp moa.jar moa.DoTask  
"EvaluatePrequential -l trees.HoeffdingTree  
-s (ArffFileStream -f GMSC_nm.arff)  
-e (BasicClassificationPerformanceEvaluator -o -r)  
-i 150000 -f 100"
```

7 Appendix

```
1 import numpy as np  
2 from collections import Counter  
3  
4 class LLMClassWindowKNN:  
5     def __init__(self, k=3, w=500):  
6         # Initialize the Class-Windowed KNN.  
7         # :param k: Number of neighbors for both local and global selection.
```

```

8      # :param w: Maximum window size (number of instances) per class.
9
10     self.k = k
11     self.w = w
12     self.X = [] # Feature vectors
13     self.y = [] # Labels
14     self.class_window = {} # Map: label -> list of indices
15
16     def train(self, instance):
17         #Adds a new instance to the model and maintains per-class window size.
18
19         idx = len(self.X)
20         self.X.append(instance.feats)
21         self.y.append(instance.label)
22
23         label = instance.label
24         if label not in self.class_window:
25             self.class_window[label] = []
26
27         self.class_window[label].append(idx)
28
29         # Enforce per-class window size (FIFO)
30         if len(self.class_window[label]) > self.w:
31             self.class_window[label].pop(0)
32
33     def predict(self, instance):
34         # Predicts the label using a two-stage k-selection process.
35
36         # top_distance will store tuples of (distance, label)
37         top_distance = []
38
39         for label, indices in self.class_window.items():
40             class_features = np.array([self.X[i] for i in indices])
41
42             distances = np.linalg.norm(class_features - instance.feats, axis=1)
43
44             local_k = min(self.k, len(distances))
45
46             k_idx_in_window = np.argpartition(distances, local_k - 1)[:local_k]
47
48             for i in k_idx_in_window:
49                 top_distance.append((distances[i], label))
50
51         if not top_distance:
52             return None
53
54         top_distance.sort(key=lambda x: x[0])
55         final_k = min(self.k, len(top_distance))
56         final_neighbors = top_distance[:final_k]
57         votes = Counter([neighbor[1] for neighbor in final_neighbors])
58         return votes.most_common(1)[0][0]
59
60 # --- Helper class for testing ---
61 class Instance:
62     def __init__(self, feats, label):
63         self.feats = feats
64         self.label = label

```

Listing 1: Buggy LLM implementation of Class-Windowed KNN using Python